

AI-BASED PHISHING URL DETECTION SYSTEM

Professional Technical Project Documentation

Author

Dushantha Sadaruwan

Project Type

Machine Learning + Cybersecurity

Repository Model

Public documentation / private implementation

Copyright © 2026 Dushantha Sadaruwan

Contents

LIST OF FIGURES	3
1. Document Purpose	4
2. Project Overview	4
3. Problem Statement	4
4. Project Scope.....	4
4.1 Included.....	4
4.2 Outside the current scope	4
5. Objectives.....	5
6. System Workflow.....	5
7. System Architecture	5
8. Development Methodology	6
9. Dataset Description	6
10. Data Preprocessing Method.....	6
10.1 Quality assessment	6
10.2 Duplicate removal	6
10.3 Label encoding	6
10.4 Train/test separation	6
11. Feature Engineering	6
11.1 Feature construction concept.....	7
12. Exploratory Data Analysis	7
12.1 Class distribution.....	7
12.2 Average URL length	7
12.3 Mean feature comparison	7
12.4 Feature-target correlation	7
12.5 Feature-to-feature correlation.....	7
13. Machine Learning Methodology	7
13.1 Logistic Regression	7
13.2 Random Forest	8
13.3 Balanced Random Forest configuration	8
14. Training and Testing	8
15. Evaluation Methodology	8
16. Recorded Experimental Results	8
16.1 Logistic Regression — cleaned-data experiment	8
16.2 Standard Random Forest — cleaned-data experiment	8
16.3 Balanced Random Forest — later experiment.....	9
17. Confusion Matrix Analysis.....	9
18. Sample Prediction Workflow	9

19. Model Persistence..... 9

20. Code and Implementation Disclosure Policy 9

21. Security and Privacy Considerations..... 10

22. Limitations 10

23. Future Improvements 10

24. Reproducibility..... 10

25. Experimental Consistency Note 10

26. Project Significance..... 11

27. Intellectual Property and Licensing..... 11

28. Public Repository Structure..... 11

29. Conclusion..... 11

Appendix A — Key Recorded Project Facts..... 12

 30.1 Project Summary Table 12

 30.2 Recorded Model Results..... 12

31. Project Figures and Visual Analysis..... 13

32. Feature Definition Reference 15

LIST OF FIGURES

- Figure 1. Dataset label distribution
- Figure 2. Mean feature values by label
- Figure 3. Logistic Regression confusion matrix
- Figure 4. Project workflow

1. Document Purpose

This document provides a professional technical description of the AI-Based Phishing URL Detection System. It explains the problem, scope, methodology, workflow, data processing, feature engineering, machine-learning methods, evaluation approach, recorded experimental results, limitations, security considerations, and future development.

This is public-facing documentation. It intentionally does not disclose the complete original source code, private notebook implementation, trained model file, or full dataset.

2. Project Overview

Phishing attacks commonly use deceptive URLs to direct users toward malicious destinations. This project investigates whether simple lexical characteristics of a URL can be transformed into numerical features and used by machine-learning classifiers to distinguish phishing-oriented URLs from legitimate URLs.

The implementation uses five URL-level features and evaluates classical machine-learning approaches. It is a practical cybersecurity and machine-learning foundation rather than a complete real-time phishing protection platform.

3. Problem Statement

Manual inspection of URLs is difficult at scale, and a URL can contain structural patterns that are not immediately obvious to a user. A lightweight classifier can provide an additional automated signal by analysing measurable properties of the URL string.

The central problem is to investigate how effectively a small set of lexical URL features can support binary classification of URLs into phishing-oriented and legitimate classes.

4. Project Scope

4.1 Included

- Dataset quality assessment and exploratory analysis.
- Duplicate removal and label encoding.
- Extraction of five lexical URL features.
- Exploratory analysis of feature distributions and correlations.
- Training and evaluation of Logistic Regression and Random Forest models.
- Investigation of class weighting with Random Forest.
- Confusion-matrix and classification-metric analysis.
- Demonstration of single-URL feature construction and prediction.
- Private model persistence using a serialized model artifact.

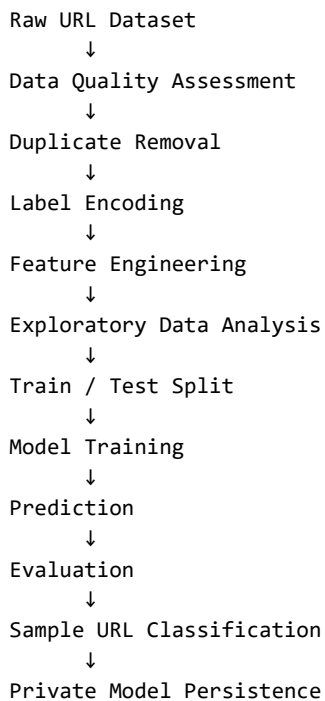
4.2 Outside the current scope

- Web-page HTML or JavaScript inspection.
- DNS and WHOIS intelligence.
- Domain reputation services.
- Browser-level blocking.
- Real-time threat-intelligence feeds.
- Large-scale deep-learning URL or webpage models.
- Automated SOC containment.

5. Objectives

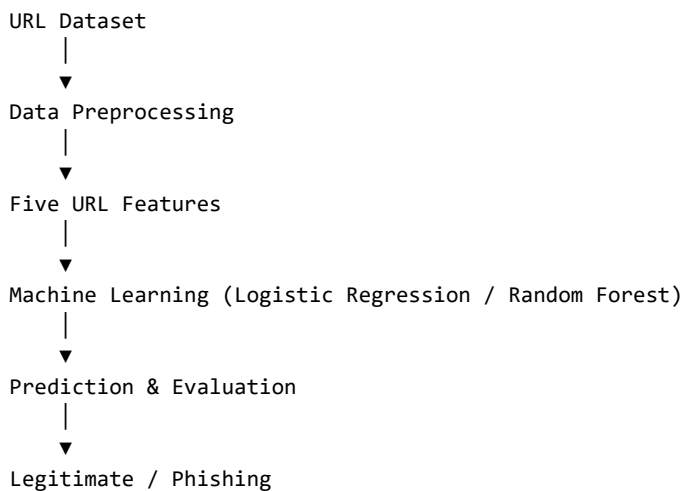
- Prepare and inspect a large labelled URL dataset.
- Remove duplicate observations and construct numerical URL features.
- Understand feature behaviour through exploratory data analysis.
- Build baseline and tree-based machine-learning classifiers.
- Measure performance using multiple metrics rather than accuracy alone.
- Study the effect of class imbalance handling.
- Create a reproducible workflow for future security-oriented extensions.

6. System Workflow



The workflow separates data preparation, feature construction, model development, and evaluation so that each stage can be reviewed and improved independently

7. System Architecture



8. Development Methodology

The project follows an experimental supervised-machine-learning methodology. The raw dataset is first inspected for structure, missing values, duplicate observations, and class distribution. URL strings are then converted into numerical features. Exploratory analysis is used to understand feature behaviour before supervised models are trained.

A linear baseline and tree-based classifiers are evaluated with precision, recall, F1-score, accuracy, and confusion matrices. Recorded experiments are interpreted together with the dataset state used for each experiment.

9. Dataset Description

Item	Recorded value
Initial records	549,346
Columns	URL, Label
Missing URL values	0
Missing Label values	0
Duplicate rows	42,150
Rows after duplicate removal	507,196
Original labels	good / bad
Model target encoding	good → 0, bad → 1

The categorical target was encoded numerically for supervised learning. Class 1 represents the phishing-oriented 'bad' class and class 0 represents the legitimate 'good' class.

10. Data Preprocessing Method

10.1 Quality assessment

The dataset was checked for missing values and duplicate rows. The initial inspection recorded no missing values in the URL and Label columns and 42,150 duplicate rows.

10.2 Duplicate removal

Duplicate observations were removed before the main cleaned-data modelling experiment, producing 507,196 records.

10.3 Label encoding

The labels were encoded as good = 0 and bad = 1.

10.4 Train/test separation

The recorded split used test_size=0.2 and random_state=42. For future controlled experiments, stratified splitting is recommended because the target classes are imbalanced.

11. Feature Engineering

The final model used exactly five URL-level features. They can be calculated directly from the URL string without fetching or executing the destination webpage.

Feature	Meaning	Type
num_dots	Number of '.' characters	Count
num_hyphens	Number of '-' characters	Count
num_at	Number of '@' characters	Count
num_digits	Number of numeric characters	Count
url_length	Total number of URL characters	Integer

11.1 Feature construction concept

For each URL:

- calculate character counts
- calculate total string length
- store the values as numerical model features

This is a lexical feature-engineering approach. It is lightweight because it does not require downloading webpage content.

12. Exploratory Data Analysis

12.1 Class distribution

Class	Count	Approx. share
good	392,924	71.53%
bad	156,422	28.47%

The target distribution is imbalanced. Therefore, a strong overall accuracy can coexist with weak minority-class detection.

12.2 Average URL length

Label	Mean URL length
bad	63.217597
good	45.767512

The recorded average URL length was higher for the bad class by approximately 17.45 characters. This is a descriptive dataset observation, not evidence of causation.

12.3 Mean feature comparison

Label	url_length	num_dots	num_hyphens	num_at	num_digits
bad	63.217597	2.766817	0.609505	0.011904	8.878911
good	45.767512	1.781856	1.332260	0.000682	3.133372

12.4 Feature-target correlation

Feature	Correlation with Label_num
num_dots	0.293474
num_hyphens	-0.134541
num_at	0.074363
num_digits	0.224053
url_length	0.177295

The strongest positive feature-target correlation recorded was num_dots. Correlation measures linear association and is not proof of causation or a complete measure of model importance.

12.5 Feature-to-feature correlation

The strongest recorded feature-to-feature correlation was num_digits versus url_length at 0.772517. This indicates substantial linear association and is relevant when considering feature redundancy.

13. Machine Learning Methodology

13.1 Logistic Regression

Logistic Regression estimates the probability of a binary class from a weighted combination of input features. It provides a useful, lightweight baseline and is comparatively straightforward to interpret.

13.2 Random Forest

Random Forest combines multiple decision trees and aggregates their predictions. The ensemble can model nonlinear relationships and feature interactions.

13.3 Balanced Random Forest configuration

The balanced configuration uses `class_weight='balanced'` to investigate whether explicit class weighting changes minority-class behaviour.

14. Training and Testing

The recorded modelling workflow separates feature matrix `X` from target vector `y` and uses a 20% test set with `random_state=42`.

Conceptual structure:

`X` = five engineered URL features

`y` = encoded target

training data → model fitting

test data → unseen evaluation

For future tuning, cross-validation or a validation set should be used while preserving a final hold-out test set.

15. Evaluation Methodology

Metric	Meaning
Accuracy	Proportion of all predictions that are correct.
Precision	Among predicted positive cases, proportion actually positive.
Recall	Among actual positive cases, proportion correctly detected.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Counts of TN, FP, FN, and TP.
Macro average	Unweighted average of per-class metrics.
Weighted average	Average weighted by class support.

For phishing detection, recall for the phishing class is particularly important because a false negative is a phishing-oriented URL classified as legitimate. Precision is also important because excessive false positives can reduce trust in an automated detector.

16. Recorded Experimental Results

16.1 Logistic Regression — cleaned-data experiment

Class	Precision	Recall	F1
0	0.83	0.98	0.90
1	0.82	0.28	0.42

Recorded accuracy: 0.8254929022082019 (approximately 82.55%). The phishing-oriented class had substantially lower recall than class 0.

16.2 Standard Random Forest — cleaned-data experiment

Class	Precision	Recall	F1
0	0.86	0.97	0.91
1	0.79	0.46	0.58

Recorded accuracy: approximately 0.85. Confusion matrix: TN=76,043, FP=2,693, FN=12,279, TP=10,425.

16.3 Balanced Random Forest — later experiment

Class	Precision	Recall	F1
0	0.86	0.86	0.86
1	0.64	0.64	0.64

Recorded accuracy: approximately 0.79. This experiment followed a reload of the original dataset and therefore should not be treated as a directly controlled comparison against the cleaned-data Random Forest experiment.

17. Confusion Matrix Analysis

For the later 109,870-row test-set state, the recorded Logistic Regression confusion matrix was:

	Predicted 0	Predicted 1
Actual 0	76,417	2,253
Actual 1	22,481	8,719

TN=76,417, FP=2,253, FN=22,481, TP=8,719. The high false-negative count demonstrates why accuracy alone is insufficient for security-oriented classification.

The standard Random Forest experiment on the 101,440-row test set recorded TN=76,043, FP=2,693, FN=12,279, TP=10,425. These matrices belong to different experiment states and should not be merged.

18. Sample Prediction Workflow

A sample URL was converted into the same five feature types used during training and passed to the trained classifier. The recorded sample URL was classified as Legitimate.

URL

- extract five numerical features
- construct one feature row
- pass row to trained classifier
- obtain class prediction

The original prediction script is intentionally not included in this public document.

19. Model Persistence

The trained model was serialized using joblib into a .pkl artifact during the private implementation. The model artifact is intentionally excluded from the public repository.

Because the notebook variable `rf_model` was reused between standard and balanced Random Forest experiments, the exact model variant associated with the final saved artifact should be explicitly verified before a new release. Future experiments should use distinct model variable and filename conventions.

20. Code and Implementation Disclosure Policy

- Public documentation contains methodology, explanations, selected results, and non-sensitive conceptual examples.
- The complete original Python source remains private.
- The full notebook remains private.
- The trained model artifact remains private.
- The full dataset remains private or subject to its original distribution terms.
- The public document is not intended to be a substitute for the private implementation.

21. Security and Privacy Considerations

- Do not place credentials, API keys, private URLs, or internal infrastructure details in the public repository.
- Do not upload the trained model if it is intended to remain private.
- Do not upload datasets whose redistribution is restricted.
- Use .gitignore to reduce accidental future tracking of private artifacts.
- .gitignore does not erase files already committed to Git history.
- Review repository history if sensitive files were previously committed.

22. Limitations

- The model relies on only five lexical URL features.
- URL-level features cannot directly inspect webpage content or behaviour.
- The recorded dataset may not represent current real-world phishing campaigns.
- Class imbalance affects minority-class detection.
- The experiments do not establish performance against live threat-intelligence feeds.
- Results from different notebook states are not a controlled benchmark unless the same dataset snapshot and split are reused.
- External validation on an unseen contemporary dataset is required before operational deployment.

23. Future Improvements

- Use stratified cross-validation and a final untouched hold-out test set.
- Perform systematic hyperparameter tuning.
- Expand URL features with validated lexical and host-based indicators.
- Evaluate calibrated probabilities and decision thresholds.
- Add explainability methods such as feature contribution analysis.
- Evaluate on temporally separated datasets to measure generalization.
- Investigate advanced machine-learning and deep-learning approaches.
- Integrate threat-intelligence signals in a controlled defensive architecture.
- Develop a secure real-time service only after rigorous validation.
- Study robustness against adversarial URL manipulation.

24. Reproducibility

- Keep the exact dataset version used for each experiment.
- Record feature definitions and preprocessing order.
- Record random seeds and split strategy.
- Record library versions in a requirements/environment file.
- Use separate model filenames for different experimental configurations.
- Keep an experiment log linking each result to its dataset state and code version.

25. Experimental Consistency Note

The notebook reloaded the original 549,346-row dataset after an earlier cleaned-data experiment. Consequently, some later results use a 109,870-row test set while earlier cleaned-data results use a 101,440-row test set. The recorded results are preserved as experiment history rather than presented as one unified controlled benchmark.

For a future final benchmark, create one fixed cleaned dataset, perform one reproducible stratified split, train all candidate models on exactly the same training data, and evaluate them on exactly the same untouched test data.

26. Project Significance

This project demonstrates a practical intersection of cybersecurity and machine learning, covering dataset analysis, feature engineering, supervised classification, class-imbalance awareness, evaluation, and secure handling of implementation artifacts.

As a foundational project, it provides a technical base for more advanced work involving explainable detection, behavioural analytics, threat intelligence, and automated defensive response.

27. Intellectual Property and Licensing

The public repository uses a proprietary, all-rights-reserved project license. The license reserves rights over original project materials and does not replace the separate licenses of third-party libraries, datasets, frameworks, or external resources.

Copyright © 2026 Dushantha Sadaruwan. All rights reserved.

28. Public Repository Structure

```
Phishing-URL-Detector/
├── README.md
├── LICENSE
├── .gitignore
├── .gitattributes
├── docs/
│   └── PROJECT_DOCUMENTATION.md
```

The public repository intentionally excludes the complete source code, private notebook, trained model, and full dataset.

29. Conclusion

The AI-Based Phishing URL Detection System demonstrates how a compact set of lexical URL features can be transformed into machine-learning features and used to investigate automated phishing classification with classical supervised learning methods. The project established a complete experimental workflow covering dataset inspection, duplicate removal, feature engineering, exploratory data analysis, model training, evaluation, and single-URL prediction.

The recorded experiments demonstrate meaningful differences between the evaluated model configurations. The Random Forest experiment achieved higher phishing-class recall than the recorded Logistic Regression experiment, while the balanced Random Forest experiment produced a more balanced class-level performance profile under a different experimental dataset state. These results should be interpreted as documented project experiments rather than as a final production security benchmark. The findings also demonstrate why phishing-class precision, recall, F1-score, and confusion-matrix analysis are important alongside overall accuracy.

The project also highlights an important limitation: URL lexical features alone cannot fully capture modern phishing behaviour. Before production use, the system would require controlled benchmarking, contemporary external datasets, stronger validation, robustness testing, security review, and an appropriate deployment architecture.

Future development can extend the system with richer URL parsing, domain and certificate intelligence, content-based signals, explainable AI, adversarial testing, calibrated risk scoring, and evaluation using temporally separated datasets.

Overall, this project provides a practical foundation for further cybersecurity research while keeping the complete implementation, trained model, notebook, and dataset outside the public repository.

Appendix A — Key Recorded Project Facts

Category	Recorded information
Project	AI-Based Phishing URL Detection System Using Machine Learning
Dataset	phishing_site_urls.csv
Initial size	549,346 rows
Duplicates	42,150
Cleaned size	507,196 rows
Features	num_dots, num_hyphens, num_at, num_digits, url_length
Target	good=0, bad=1
Libraries	pandas, NumPy, matplotlib, seaborn, scikit-learn, joblib
Primary models	Logistic Regression, Random Forest
Class-balanced experiment	Random Forest with class_weight='balanced'
Public source policy	Original implementation remains private

30.1 Project Summary Table

Item	Recorded Project Information
Project title	AI-Based Phishing URL Detection System Using Machine Learning
Initial dataset size	549,346 URLs
Duplicate rows	42,150
Cleaned dataset size	507,196 rows
Input features	5 URL-level lexical features
Primary models	Logistic Regression; Random Forest
Balanced model	Random Forest with class_weight='balanced'
Libraries	pandas, NumPy, matplotlib, seaborn, scikit-learn, joblib
Implementation disclosure	Complete source/model files kept private

30.2 Recorded Model Results

Experiment	Accuracy	Class 1 Precision	Class 1 Recall	Class 1 F1	Test Support
Logistic Regression – cleaned 101,440 test set	0.83	0.82	0.28	0.42	101,440
Logistic Regression – reloaded 109,870 test set	0.77	0.79	0.28	0.41	109,870
Random Forest – cleaned 101,440 test set	0.85	0.79	0.46	0.58	101,440
Balanced Random Forest – reloaded 109,870 test set	0.79	0.64	0.64	0.64	109,870

31. Project Figures and Visual Analysis

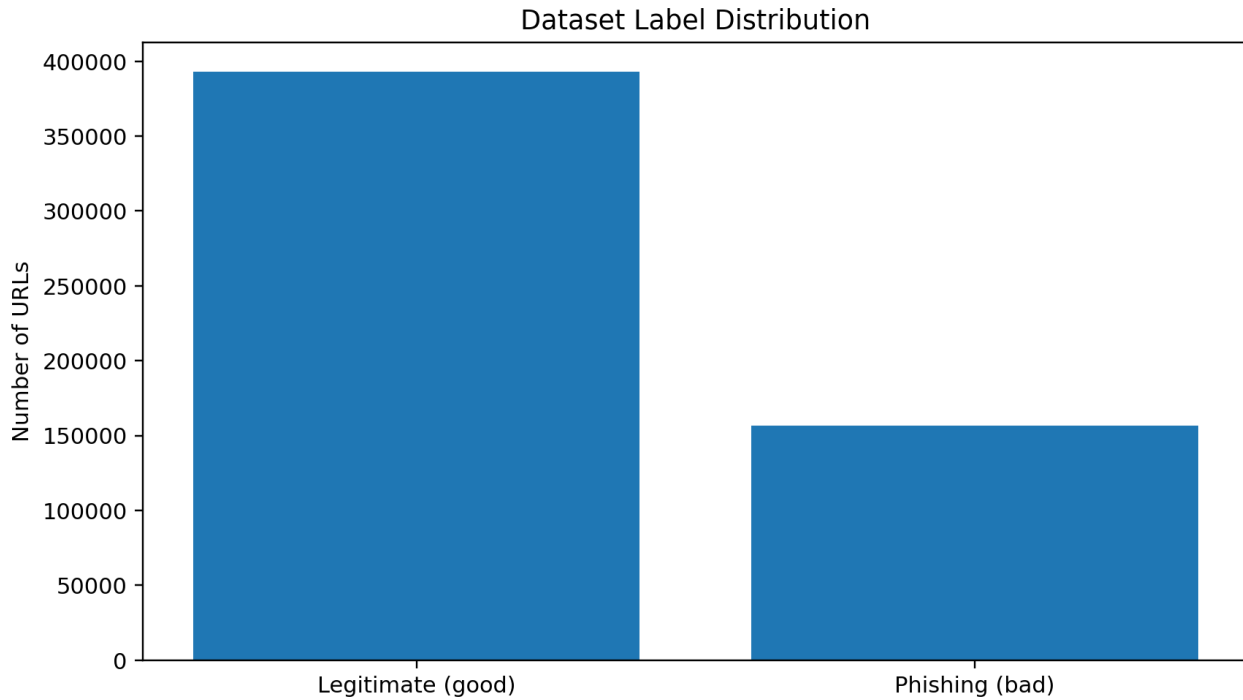


Figure 1. Dataset label distribution

The original dataset contains 392,924 good URLs and 156,422 bad URLs.

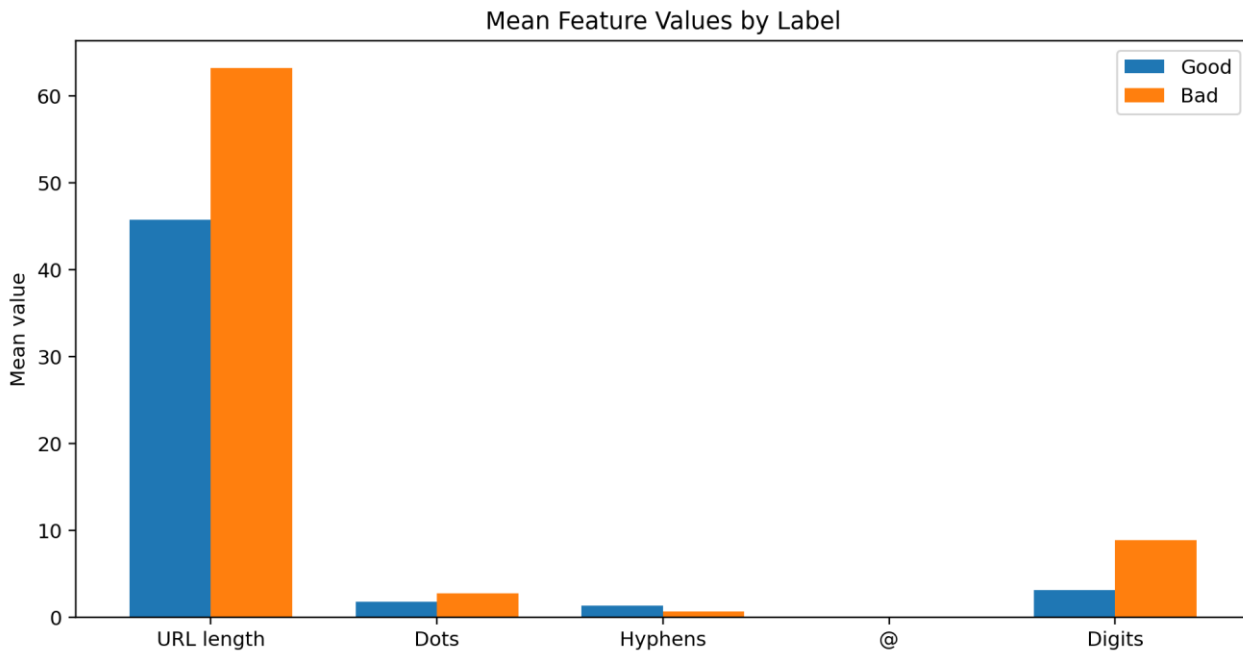


Figure 2. Mean feature values by label

Recorded group means for the five engineered URL-level features.

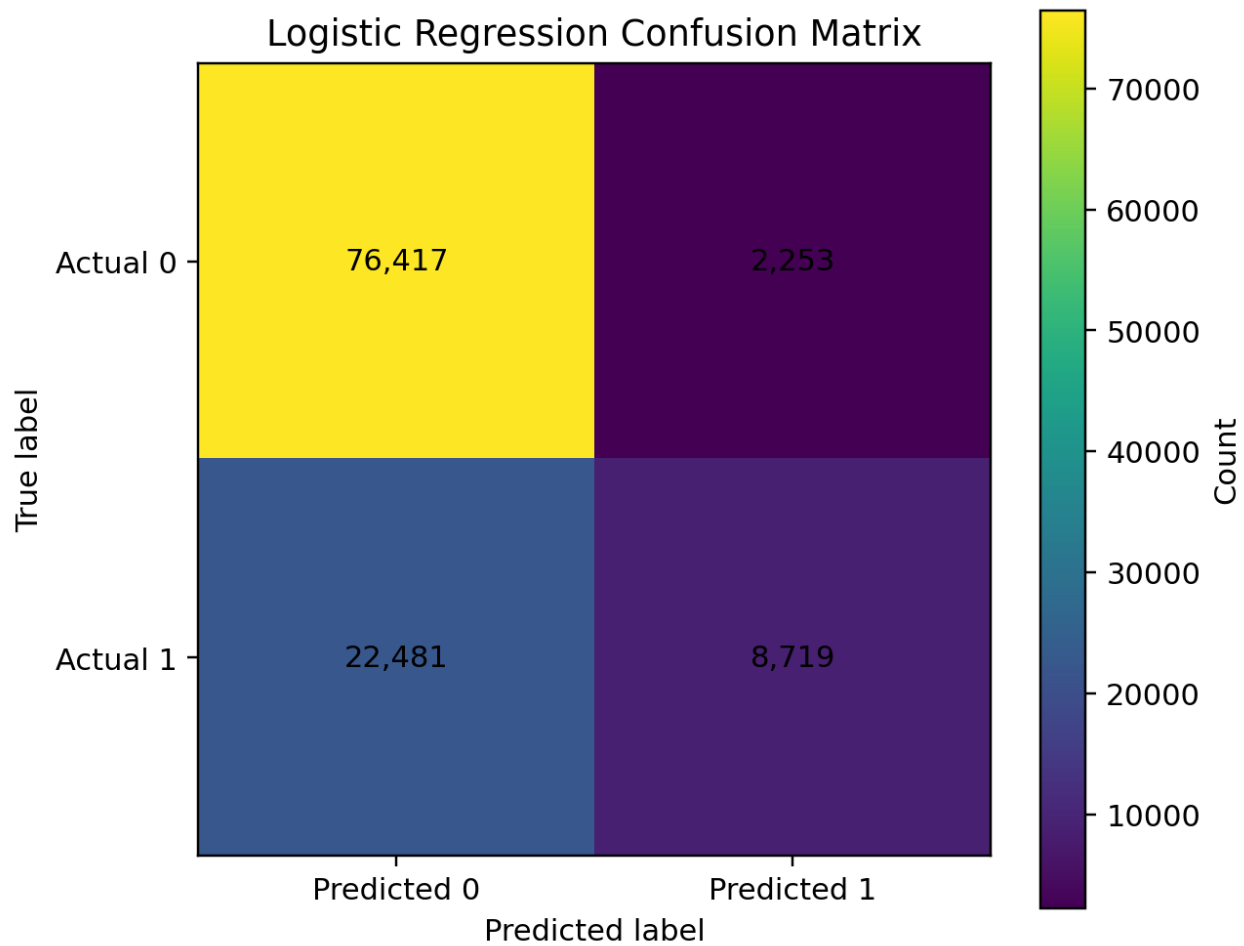


Figure 3. Logistic Regression confusion matrix

Recorded confusion matrix from the reloaded Logistic Regression experiment: $TN=76,417$, $FP=2,253$, $FN=22,481$, $TP=8,719$.

Project Workflow

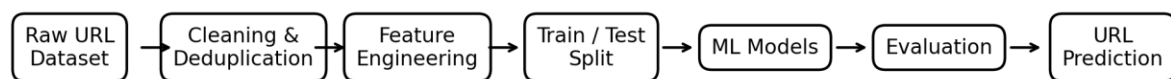


Figure 4. Project workflow

High-level project workflow from raw URL data through preprocessing, feature engineering, model training, evaluation, and prediction.

32. Feature Definition Reference

Feature	Definition	Purpose in Experiment
num_dots	Count of '.' characters in the URL	Captures domain/subdomain/path structural complexity
num_hyphens	Count of '-' characters	Captures hyphen usage in the URL
num_at	Count of '@' characters	Captures use of an unusual URL character
num_digits	Count of numeric digits	Captures numeric density in the URL
url_length	Total number of characters in the URL	Captures overall URL length